

Comparative Study between Genetic Algorithm and Heuristic Algorithms on Benchmark Optimization Functions

Obreja Rares

November 24, 2025

Abstract

We designed a Genetic Algorithm that uses a population of candidate solutions and a selection process applied to it throughout multiple generations and compared the results obtained with the ones gotten from Hill Climbing Algorithms and Simulated Annealing on the same benchmark numerical functions. In the end the Genetic Algorithm yielded much better results on Rastrigin's and Schwefel's functions and a slightly better result on Michalewicz's function while on De Jong's Sphere Model also found the optimum just like HC Best Improvement, but also the average run-time is significantly longer. The results on Rastrigin's and Schwefel's functions were really surprising and made a huge step towards the optimum compared to the results that we obtained from the other algorithms. For instance for Rastrigin's function the mean of the results went from barely under 30 to under 5 and for Schwefel's function we went from an average of 1512 to an impressive 1.75. The Genetic Algorithm performed way better than the rest of the algorithms we used and it surely can be fine-tuned to get even better results with the right parameters set.

1 Introduction

The Genetic Algorithm used is an extension of Hill Climb Best Improvement Algorithm that uses a population of solutions instead of just one and through crossover and mutation selects future generations in hope of finding the optimum solution within the population. The selection process can be optimized by modifying the way we calculate the fitness function for each candidate therefore, we can have a more relaxed constraint so that slightly worse solutions also have a good change of being selected, or we can be very rigid and only allow the best ones in the next generation with results in an elitism approach. Another important aspect is the number of crossovers and mutations we allow which can give a huge exploration within the population or making it remain basically unchanged after these processes. The purpose

of this paper is the comparative study of this Genetic Algorithm with Hill Climbing algorithms and Simulated Annealing to optimize some well-known multidimensional mathematical functions (De Jong's, Schwefel, Rastrigin, and Michalewicz). The implemented algorithms are evaluated based on their convergence behavior and robustness in finding near optimal solutions within bounded search spaces.

2 Algorithms used and their implementation

2.1 Hill Climbing

Hill Climbing is an iterative search algorithm that starts from a random candidate solution and looks through its neighbors in order to find a better solution. Here we are comparing 3 versions of it: Best, First and Worst improvement. One downside of it is that it is more prone to get stuck in a local optima and not get to the actual global one.

2.2 Simulated Annealing

Simulated Annealing is based on a method used in metallurgy that heats a metal and modifies it while the temperature decreases. The algorithm is a modified version of Hill Climbing Best Improvement which allows a worse solution to be taken with a small probability. The probability depends on the temperature, and as the algorithm progresses, the temperature is cooling and so the chance to pick a worse solution is getting closer to 0. This method allows the solution to escape a possible local minima in the hope of finding the global one and usually has a longer runtime.

2.3 Genetic Algorithm

A genetic algorithm (GA) is a computational method inspired by natural selection to find approximate solutions for optimization and search problems. It involves creating a population of potential solutions, evaluating their fitness using an objective function, and then using operators like selection, crossover, and mutation to create new, fitter generations. This process repeats until a satisfactory solution is found.

In our implementation we started with a random population that we improved by applying HC Best Improvement to each candidate so we start with a better population than just random candidates. Then we evaluated it with a fitness function so that better results (smaller values on the numerical function evaluation) had a bigger fitness in order to have a better chance of being selected for the next generation through the wheel of fortune method. After that we choose the ones that will undergo crossover. From those we select pairs of two that in return will give two offsprings. For each of the offsprings we apply HC Best Improvement in order to fine-tune them into better solutions. In the end we apply mutation so that each bit in each candidate solution has an equal chance of being selected and we mutate it by flipping its value.

2.3.1 Implementation

- Solutions - represented as a vector or bits containing all the dimension values one after another (each values is represented on 30 bits - the whole solution has at most 900 bits in the case of 30 dimensions)
- Population Size - 100
- Crossover probability - 0.4
- Mutation probability - 0.002
- Selection pressure (in order to give better chances at being selected toward the better candidates) - $\text{best_of_gen} / 2$ (best_of_gen for Michalewicz's function also for turning the values positive)
- Fitness function - $1.0 / (f(x_i) - \text{selection_pressure} + 1)$

3 Functions Used

3.1 De Jong (sphere model)

De Jong's function is continuous, convex and uni-modal with a global minimum of 0 that is reached when the value for each dimension is also 0.

Function definition:

$$f(x) = \sum_1^n (i \cdot x_i^2), \quad -5.12 \leq x_i \leq 5.12.$$

3.2 Rastrigin

Rastrigin's function is highly multimodal. However, the location of the minima are regularly distributed with the global minimum value being 0 that is reached when the value for each dimension is also 0, just as De Jong's function.

Function definition:

$$f(x) = 10 \cdot n + \sum_1^n (x_i^2 - 10 \cdot \cos(2 \cdot \pi \cdot x_i)), \quad -5.12 \leq x_i \leq 5.12.$$

3.3 Schwefel

Schwefel's function is deceptive in that the global minimum is geometrically distant, over the parameter space, from the next best local minima. Therefore, the search algorithms are potentially prone to convergence in the wrong direction. On our test we used the version that has 0 as the global minimum.

Function definition:

$$f(x) = \sum_1^n (-x_i \cdot \sin(\sqrt{|x_i|})) + n \cdot 418.9829, \quad -500 \leq x_i \leq 500.$$

3.4 Michalewicz

Michalewicz's function is a multimodal test function ($n!$ local optima). The parameter m defines the "steepness" of the valleys or edges. Larger m leads to more difficult search. For very large m the function behaves like a needle in the haystack (the function values for points in the space outside the narrow peaks give very little information on the location of the global optimum).

Function definition:

$$f(x) = - \sum_1^n \sin(x_i) \cdot \left(\sin\left(\frac{i \cdot x_i^2}{\pi}\right)\right)^{2 \cdot m}, \quad 0 \leq x_i \leq \pi$$

4 Experimental Results

For Hill Climbing and Simulated Annealing functions, we run each algorithm with 200 starting candidates and for the Genetic Algorithm with a population of 100, on multiple dimensions (5, 10, 30) at least 30 times to be able to draw a robust conclusion out of the results.

The parameters used in the implementation of the algorithms were deduced through various experiments. They were optimized to give the best results in the shortest time possible. The best improvements made in Hill Climbing and Simulated Annealing were:

- The introduction of an increment when exploring the neighbors - it is chosen at random between 1 and 2 allowing a faster runtime (we explore less neighbors) and finding better solutions
- The number of steps - it is used to considerably decrease the runtime of HC Worst Improvement - on Rastring's function with 30 dimensions the runtime went from 40 min to 4 min after introducing them
- The temperature in the SA algorithm was chosen so that the initial temperature is neither too high nor too low in order to have a good chance of choosing a worse solution but not being just a random selection
- Other optimization is recalculating only the dimension where the bit was flipped to generate the neighbor instead of calculating all the dimensions again

The best improvements on the Genetic Algorithm were:

- Adding Selection Pressure to the fitness function
- Applying HC Best Improvement to the initial population
- Applying HC Best Improvement to the offsprings that result from crossover.
- Other optimization is recalculating only the dimension where the bit was flipped to generate the neighbor instead of calculating all the dimensions again

4.1 De Jong

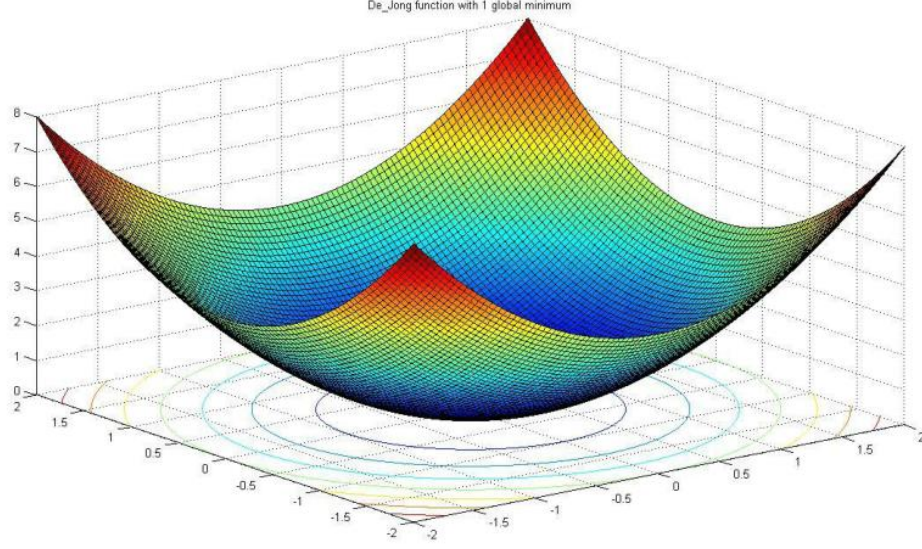


Figure 1: De Jong Sphere Model [5].

		GA	HC Best	HC First	HC Worst	SA
n=30	Min	0	0	2.74757	0.00001	0.21742
	Max	0	0	4.83235	1.68834	0.32813
	Mean	0	0	3.61304	0.22918	0.28135
	StdDev	0	0	0.45879	0.39488	0.02999
	AvgRunTime(ms)	361483	5860	2329	344047	84775
n=10	Min	0	0	0.03082	0	0.0135
	Max	0	0	0.54299	0.77312	0.03414
	Mean	0	0	0.14581	0.20418	0.02505
	StdDev	0	0	0.12987	0.23385	0.00469
	AvgRunTime(ms)	61467	785	418	11685	27471
n=5	Min	0	0	0	0	0.00026
	Max	0	0	0.00262	0.02918	0.00365
	Mean	0	0	0.00042	0.00167	0.00211
	StdDev	0	0	0.00069	0.00655	0.00088
	AvgRunTime(ms)	22309	233	166	1646	13642

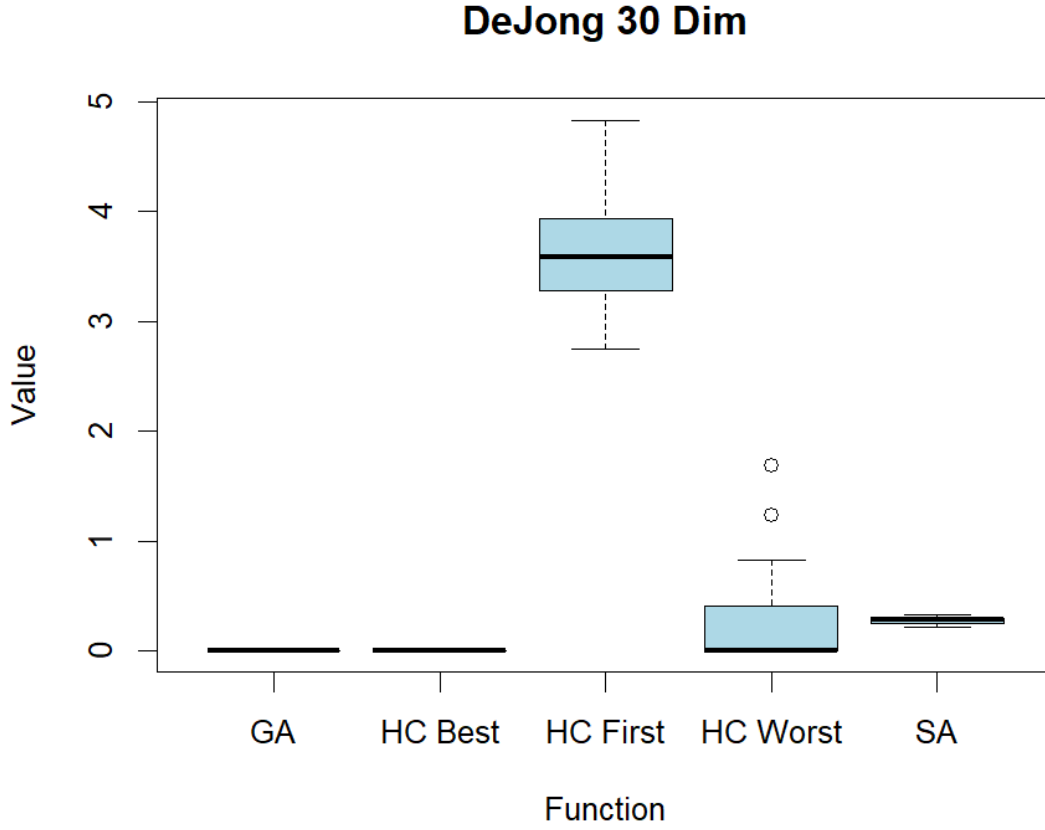
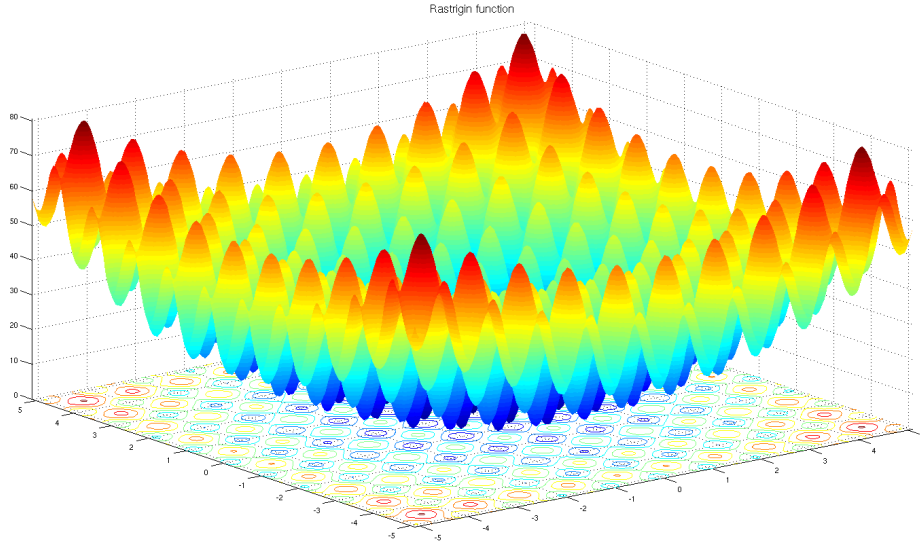


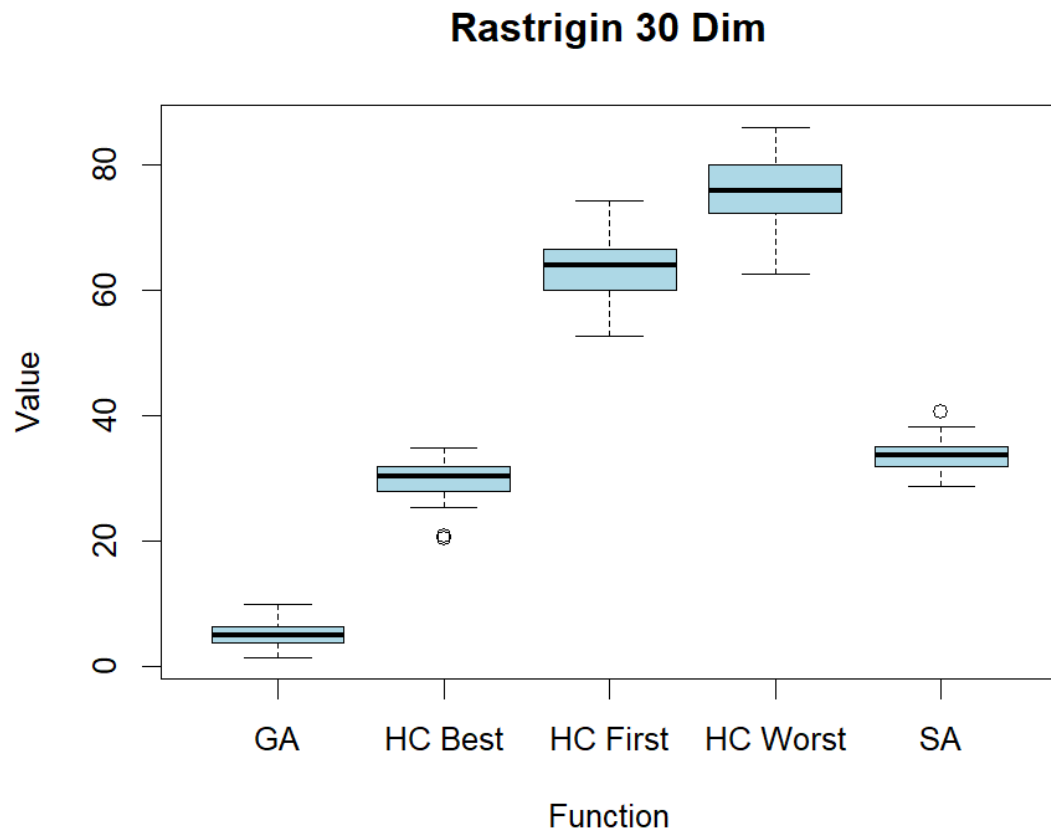
Figure 2: Rastrigin's Function [4].

On De Jong's sphere model we can observe that all the algorithms perform quite well, the worst being HC First Improvement. HC Best Improvement and the Genetic Algorithm find the minimum on every single run and HC Worst and SA get very close to it. Another observation is that while the mean results of both HC Worst and SA are close, SA has a much better convergence, having all the values really close together. Also, although HC Best Improvement and GA give the same results, GA takes a lot more time by going through a lot of generation when it probably needs only a few. This can be avoided by improving the termination condition.

4.2 Rastrigin



		GA	HC Best	HC First	HC Worst	SA
n=30	Min	1.23576	20.3613	52.58586	62.41107	28.60605
	Max	9.88614	34.7121	74.17701	85.98081	40.53051
	Mean	4.76241	29.3996	63.52755	75.7366	33.54529
	StdDev	2.0345	3.42437	5.3948	5.8839	2.4968
	AvgRunTime(ms)	341928	3730	1726	327235	113981
n=10	Min	0	0.99513	3.58726	5.03758	3.28097
	Max	2.47153	6.70727	14.60871	20.05325	7.79337
	Mean	0.17163	4.89933	10.07478	14.45382	5.81393
	StdDev	0.52452	1.27878	2.81363	3.37118	1.16509
	AvgRunTime(ms)	51803	508	300	10119	37097
n=5	Min	0	0.00001	0.02047	0.31684	0.00353
	Max	1.23576	2.23078	3.94576	6.82507	2.23728
	Mean	0.03432	1.08696	1.91354	3.27590	1.00931
	StdDev	0.20596	0.37915	0.84558	1.45527	0.65021
	AvgRunTime(ms)	18961	108	139	1439	18422



On Rastrigin's function we can see that the best results come from GA by far. Next come HC Best Improvement and SA that also have better convergence than the other two. One big difference from the results on De Jong's function is that HC Worst Improvement is now the worst one of them, and because Rastrigin is a more complex function it shows a better difference between HC Best Improvement and the GA, showing the importance of escaping a local minimum through crossover and mutation.

4.3 Schwefel

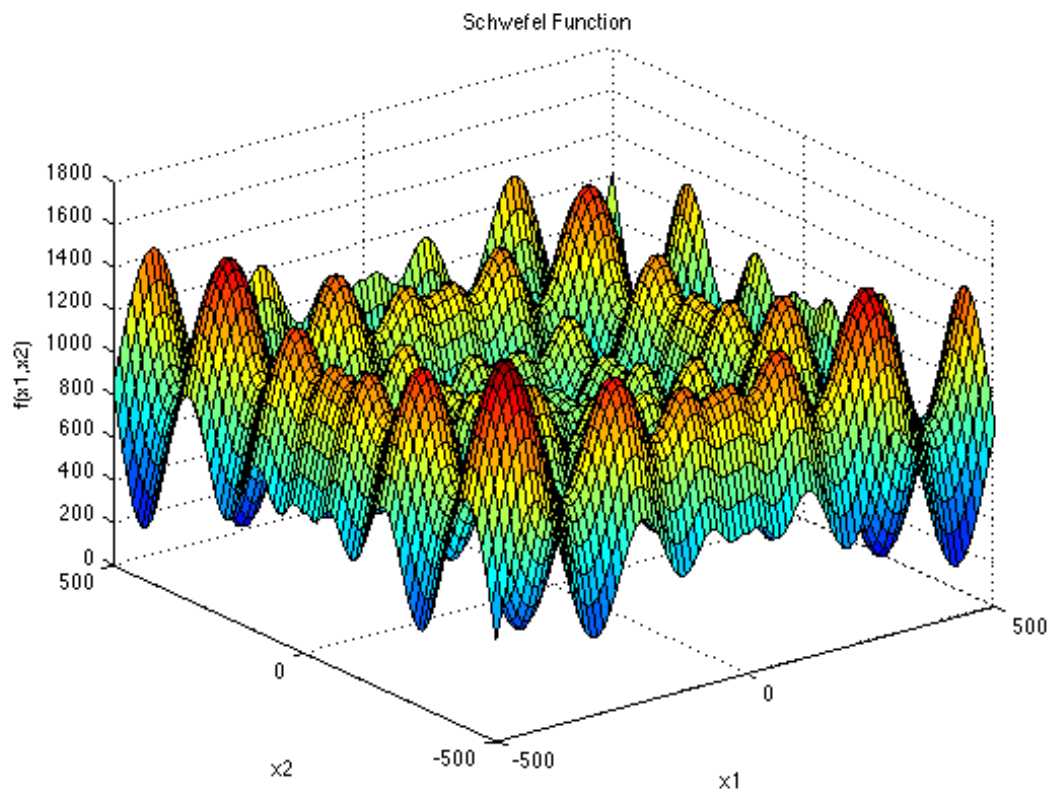
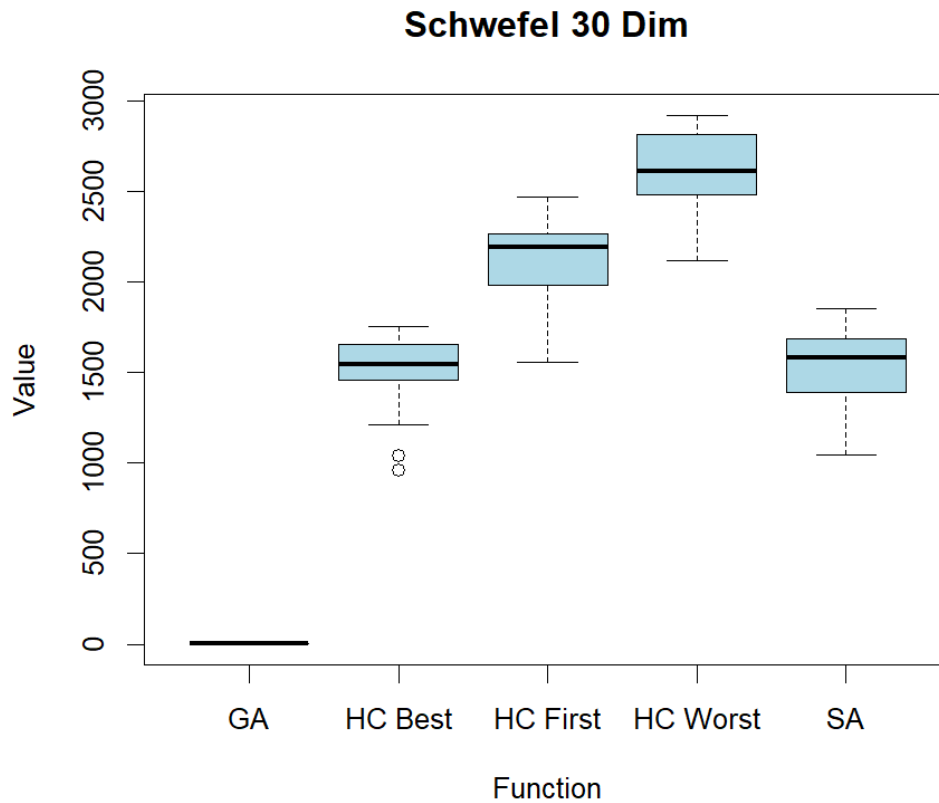
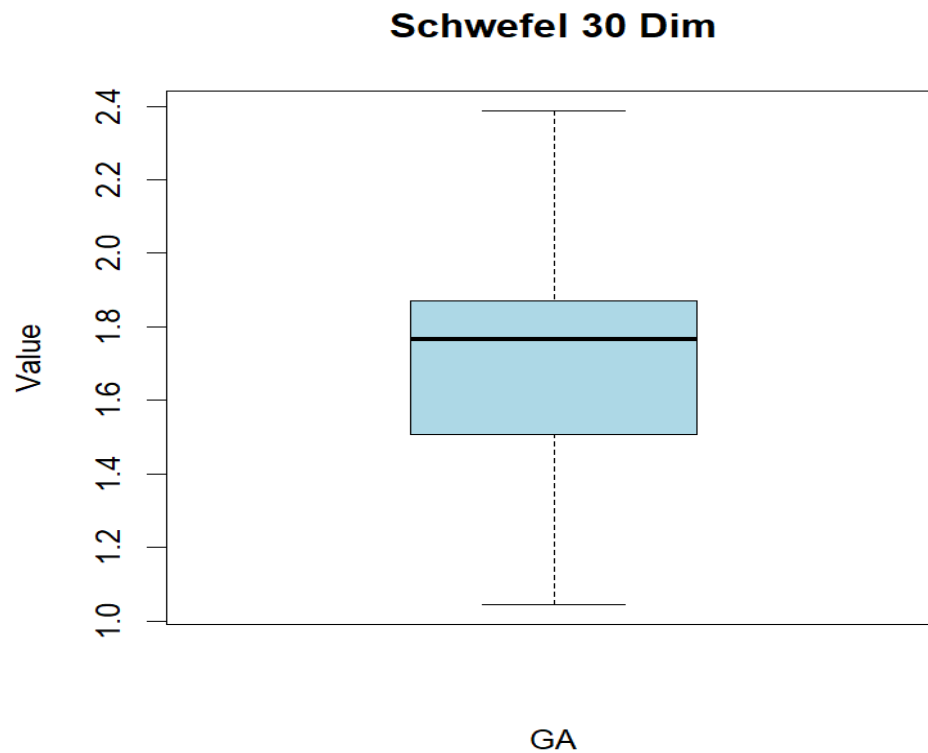


Figure 3: Schwefel's Function [6].

		GA	HC Best	HC First	HC Worst	SA
n=30	Min	1.04283	956.51876	1554.13370	2114.46600	1046.18910
	Max	2.3866	1753.80617	2466.42811	2916.28874	1851.37759
	Mean	1.74912	1512.01564	2123.46623	2608.78770	1534.08485
	StdDev	0.29184	183.57533	2123.46623	216.75917	203.70949
	AvgRunTime(ms)	374680	5716	2623	521342	78410
n=10	Min	0.10506	69.19592	171.22039	180.02901	119.52843
	Max	0.72643	425.45812	534.34671	703.90749	434.05924
	Mean	0.47925	261.89748	371.50546	485.30998	434.05924
	StdDev	0.16643	88.79442	95.08282	127.79745	72.91658
	AvgRunTime(ms)	55089	729	474	17036	24545
n=5	Min	0.00006	0.00135	0.33668	0.2303	0.04235
	Max	0.41473	88.23145	154.16473	239.25428	118.56335
	Mean	0.15238	17.52490	59.83296	115.13505	31.03361
	StdDev	0.11306	25.00089	51.23757	65.82944	33.88205
	AvgRunTime(ms)	19420	231	155	2233	11426





On Schwefel we can observe that because the values are much bigger it is harder to get close to 0 and also the values of the individual runs tend to be more spread apart. However, here we observe the biggest difference between the GA and the other algorithms. It has by far the best results, getting really close to the optimum while HC Best Improvement and SA share similar results. The results of GA are also much more concentrated showing impressive consistency throughout the 30 runs in the sample.

4.4 Michalewicz

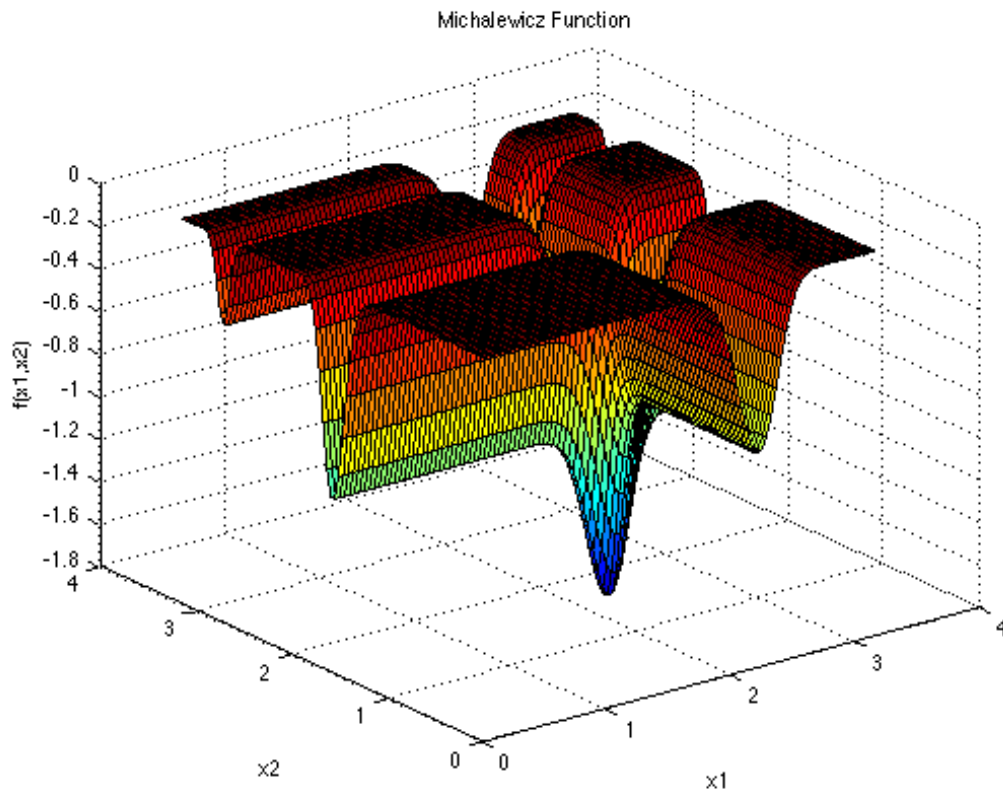
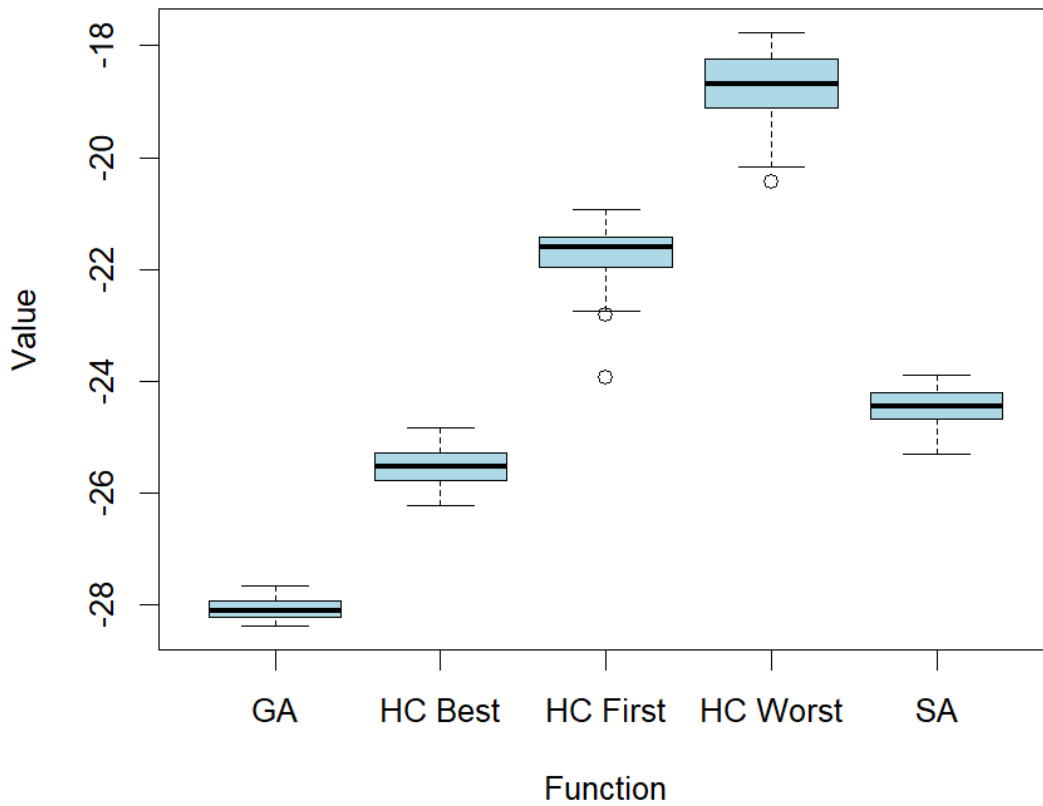


Figure 4: Michalewicz's Function [3].

		GA	HC Best	HC First	HC Worst	SA
n=30	Min	-28.39978	-26.23760	-23.93422	-20.43558	-25.32044
	Max	-27.66444	-24.84123	-20.92320	-17.77193	-23.89238
	Mean	-28.0836	-25.5027	-21.75100	-18.75593	-24.46792
	StdDev	0.1802	0.35116	0.62247	0.6795	0.36178
	AvgRunTime(ms)	400724	6649	3597	703803	99945
n=10	Min	-8.65945	-8.60635	-8.37848	-7.34091	-8.50480
	Max	-8.51013	-8.01082	-7.21282	-5.89041	-8.50480
	Mean	-8.63123	-8.30128	-7.75360	-5.89041	-8.50480
	StdDev	0.03608	0.13450	0.29322	0.36296	0.13674
	AvgRunTime(ms)	65933	907	617	16033	31229
n=5	Min	-3.69885	-3.69817	0.12606	-3.69817	-3.69885
	Max	-3.69804	-3.18608	-3.6227	-3.18608	-3.6227
	Mean	-3.69881	-3.52124	-3.68297	-3.52124	-3.68297
	StdDev	0.00016	0.12606	0.02072	0.12606	0.02072
	AvgRunTime(ms)	24403	239	207	1542	14326

Michalewicz 30 Dim



On Michalewicz's function we can observe that all the algorithms have a similar convergence and the order from other functions remains, with the GA being the best, followed by HC Best Improvement and SA which are considerably better than HC First and HC Worst Improvement. Here we used a slightly different fitness function in order to get positive values for it. We achieved this by modifying the selection pressure. In the end, it obtained better results than the other algorithms with an average of barely under -28.

5 Conclusions

In conclusion, on every function tested we obtained the best results from GA followed by HC Best Improvement and SA algorithms. They tend to have a better convergence with a less scattered distribution. The GA had a much better exploration thanks to the crossovers and mutations and could find excellent results due to fine-tuning the solutions with HC Best Improvement. The GA implemented has proved to perform really well on Schwefel's function and with some adjusting of the parameters it might be able to give similar results for the other benchmark function tested.

Another important aspect is the runtime of each algorithm. HC First Improvement is always the fastest because of the small number of neighbors checked on average, and close to it is HC Best Improvement because of the fast convergence of the values. GA is the slowest algorithm used because of the large number of processing steps it does for each generation. Also HC Worst Improvement is really slow on large number of dimensions but gets faster than SA when the number decreases, because SA has a good chance of accepting a worse solution and keep the search going.

From this experiment the best algorithm for these functions was GA by far giving the best results. HC Best Improvement also gives really good results with a much better runtime, but it doesn't come close to the GA on some of the functions. HC Worst Improvement was usually the worst at the results, but it worked really well on De Jong's function. HC First Improvement is the fastest one with better results than the Worst Improvement version but always worse than Best Improvement version. Simulated Annealing works really well on these optimization functions because it has the ability to escape a possible local minima and reach the global one, however despite having close results to HC Best Improvement it never reached better ones and the runtime is considerably longer.

The GA performed the best out of all the algorithms used and could be made even better by fine-tuning some of the parameters and also improving the termination condition in order to stop it early when the population gets saturated or when the optimum has already been found. Another idea would be to insert some new random candidates after some generations (ex: for each 100 gen) and only keep a few of the best from the previous solutions. This would increase the diversity of the population throughout the whole run and hopefully give a better chance at finding the global minimum.

References

- [1] Eugen Croitoru, *Genetic Algorithms*, Site: <https://profs.info.uaic.ro/eugen.croitoru/teaching/ga/>.
- [2] *GEATbx: Example functions*, Site:http://www.geatbx.com/docu/fcnindex-01.html#P140_6155.
- [3] *Michalewicz Function*, Site:<https://www.sfu.ca/~ssurjano/michal.html>.
- [4] *Rastrigin Function*, Site:https://en.wikipedia.org/wiki/Rastrigin_function.
- [5] *De Jong Sphere Model Function*, Site:<https://www.al-roomi.org/component/tags/tag/1st-de-jong-s-function>.
- [6] *Schwefel's Function*, Site:<https://www.sfu.ca/~ssurjano/schwef.html>.
- [7] *Hill Climbing in Artificial Intelligence*, Site:<https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/>.
- [8] *What is Simulated Annealing*, Site:<https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/>.
- [9] Chinnasamy, Sathiyaraj and Ramachandran, Amudha, Ramu, Kurinjimalar, *A review on hill climbing optimization methodology*, Recent Trends in Management and Commerce, vol.3, 2022.
- [10] Goffe, William L and Ferrier, Gary D and Rogers, John, *Global optimization of statistical functions with simulated annealing*, Journal of econometrics, vol.60, 1994.
- [11] Selman, Bart and Gomes, Carla P, *Hill-climbing search*, Encyclopedia of cognitive science, vol.81, 2006.
- [12] Charlie Vanaret, Jean-Baptiste Gotteland, Nicolas Durand, Jean-Marc Alliot *Certified Global Minima for a Benchmark of Difficult Optimization Problems*, 2014. Site:https://www.researchgate.net/publication/278769271_Certified_Global_Minima_for_a_Benchmark_of_Difficult_Optimization_Problemsz.
- [13] Pablo Moscato, Carlos Cotta *A Gentle Introduction to Memetic Algorithms(2003)*. Site:https://www.researchgate.net/publication/313051410_A_Gentle_Introduction_to_Memetic_Algorithms.